

Flink 原理详解：第 9 讲

双流匹配与 CEP：从“订单-支付配对”到复杂事件模式识别

本讲目标：理解两条实时流如何按 Key 协同处理；掌握 connect + KeyedCoProcessFunction + State + Timer 的通用模式；理解 CEP 什么时候比手写状态机更合适，并建立 Pattern / NFA 的第一层心智模型。

双流实时匹配：订单流 + 支付流

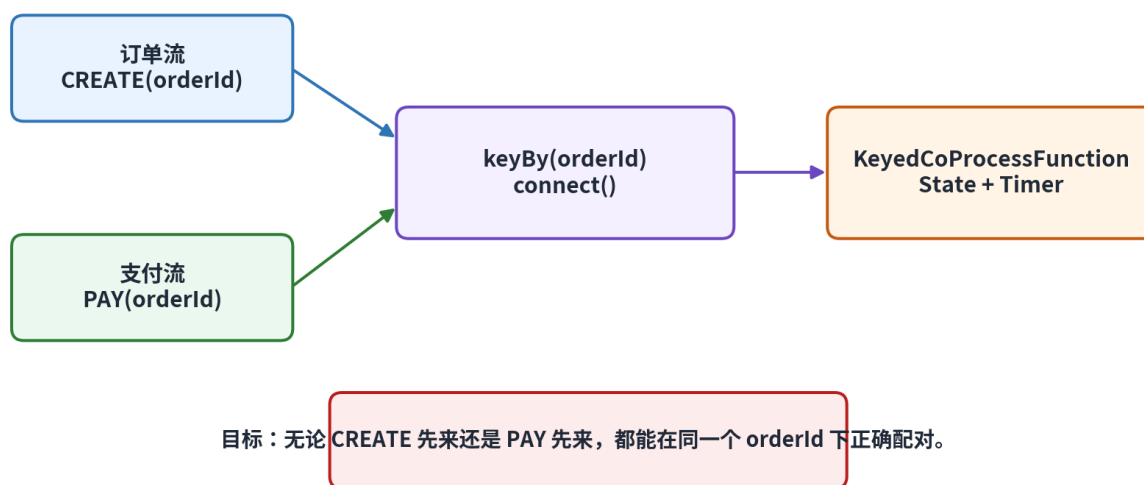


图 1 双流匹配：两条流通过相同业务 Key 汇合到同一个状态逻辑

1. 为什么“两个实时流”比单流更难？

单流规则通常只需要回答：这条事件来了，我如何结合这个 Key 的历史状态处理它。双流则多了一层不确定性：来自两个来源的相关事件，谁先到、谁后到、会不会迟到、另一边会不会永远不来，都不确定。

- 订单系统产生 CREATE(orderId=1001)。
- 支付系统产生 PAY(orderId=1001)。
- 理想情况是 CREATE 先到、PAY 后到，但真实系统可能 PAY 先到。
- 网络、Kafka 分区、上游重试都会导致两条流的到达顺序与业务发生顺序不同。

双流问题的本质不是“把两条流拼起来”，而是：如何让同一业务 Key 的两侧事件在乱序和缺失的情况下仍能正确协调。

2. union、connect、intervalJoin、CoProcess、CEP 有什么区别？

两条流“放到一起”并不只有一种方法

工具	本质	特点	典型场景
union	同类型流直接合并	无需分别识别来源	多个同构日志源
connect	不同类型流联合处理	两侧保留各自类型	订单流 + 支付流
intervalJoin	按 Key + 时间区间配对	适合明确时间范围 Join	点击 + 曝光
CoProcess	两侧事件 + State + Timer	控制力最强	乱序、超时、补偿
CEP	描述多事件时序模式	模式语言 + 状态机	风控、入侵、行为序列

选择工具的关键不是“哪个更高级”，而是：你的问题是合并、Join、状态协调，还是复杂模式识别？

图 2 不同工具解决的问题并不相同

初学者很容易看到“两条流”就想 Join。实际上，Flink 提供了多种机制，它们的抽象层次和适用问题完全不同。

机制	适合的问题	核心特点
union	多个相同类型流合并	同构流，合并后不再区分输入侧
connect	两个不同类型流联合处理	保留两侧类型，可进一步 process
intervalJoin	按 Key + 明确时间区间配对	Join 语义清晰，时间范围明确
KeyedCoProcessFunction	复杂乱序、超时、补偿逻辑	State + Timer，自由度最高
CEP	多事件时序模式识别	Pattern 描述事件序列

3. ConnectedStreams：两条流仍然保留自己的“身份”

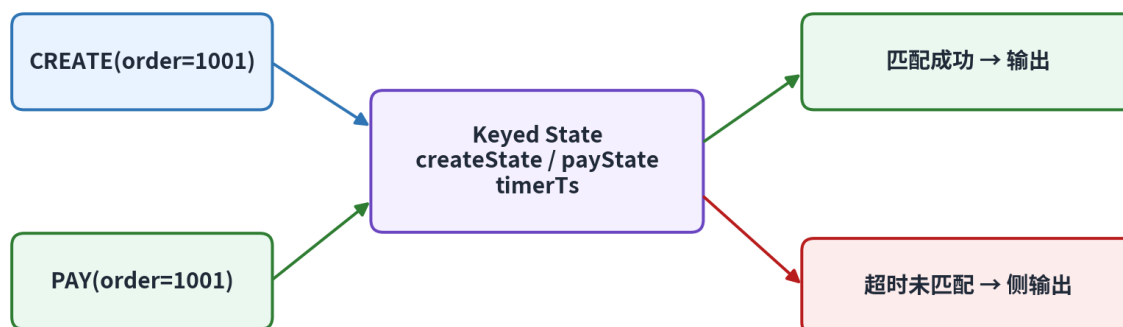
DataStream.connect() 会得到 ConnectedStreams。它与 union 最大的区别是：两侧流可以是不同类型，而且处理函数分别拥有 processElement1(...) 和 processElement2(...)。

```
ConnectedStreams<OrderEvent, PaymentEvent> connected =
    orders.keyBy(OrderEvent::getOrderId)
        .connect(payments.keyBy(PaymentEvent::getOrderId));

connected.process(new OrderPaymentMatchFunction());
```

关键约束：如果要使用 KeyedCoProcessFunction 的 keyed state / keyed timer，两侧通常都应按兼容的 Key 进行 keyBy。

CoProcess 的核心：两侧事件共享同一个 Key 的状态



关键：CREATE 和 PAY 可能任意先到，因此两边都要“查对方 State；找不到就先保存自己”。

图 3 两侧事件进入同一 KeyedCoProcessFunction 后，共享该 Key 的状态空间

4. KeyedCoProcessFunction：双输入版的 ProcessFunction

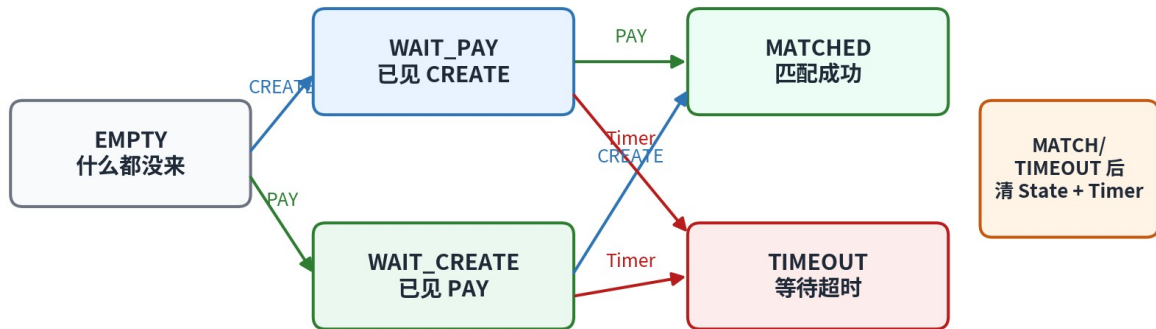
上一讲学的 KeyedProcessFunction 只有一个输入。KeyedCoProcessFunction 则为两个输入分别提供回调：

```
processElement1(IN1 value, Context ctx, Collector<OUT> out)
processElement2(IN2 value, Context ctx, Collector<OUT> out)
onTimer(long timestamp, OnTimerContext ctx, Collector<OUT> out)
```

官方文档对低层双流 Join 的建议正是：为一侧或两侧维护状态；一侧事件到来就更新状态；另一侧到来时查询状态并输出；若关心乱序和完整性，可以结合 Timer / Watermark 延后判断。

5. 案例：订单 CREATE 与 PAY 实时匹配

订单-支付匹配：一个小型双输入状态机



状态机不是额外框架：它只是我们对 State + 两侧事件 + Timer 行为的清晰表达。

图 4 双流匹配可以看成是一个按 orderId 隔离的小状态机

我们希望实现以下规则：

1. CREATE 和 PAY 都到了：输出“匹配成功”。
2. CREATE 到了，但 15 分钟后还没有 PAY：输出“未支付超时”。
3. PAY 先到了：先保存 PAY，等待 CREATE。
4. 匹配或超时以后：清理 State 和无用 Timer，避免状态无限增长。

6. 为什么两边都要“先查对方，再保存自己”？

假设 CREATE 先到：

```
processElement1(create):
  if payState != null:
    emit MATCH
    clear all state/timer
  else:
    createState = create
    register timeout timer
```

而 PAY 先到时逻辑必须镜像：

```
processElement2(pay):
  if createState != null:
    emit MATCH
    clear all state/timer
  else:
    payState = pay
    register cleanup/timeout timer
```

这就是乱序友好的核心：不要假设“正确顺序一定先到”。程序必须允许两种到达顺序。

7. Timer 在双流匹配里解决什么？

如果只保存 State 而没有 Timer，一个永远匹配不到的订单可能永远留在状态里。Timer 给了系统一个“最终裁决时刻”。例如以 CREATE 的 Event Time + 15 分钟注册 Event-Time Timer，当 Watermark 越过该时刻仍未见 PAY，就判定超时。

State 负责等待，Timer 负责结束等待。没有 Timer 的等待逻辑很容易变成“永远等下去”。

8. 什么时候 intervalJoin 更简单？

如果业务只是“同一用户的点击和购买，只要发生时间相差 -5 到 +10 分钟就配对”，并且 Join 边界非常明确，那么 intervalJoin 往往比手写 CoProcess 状态机更直接。

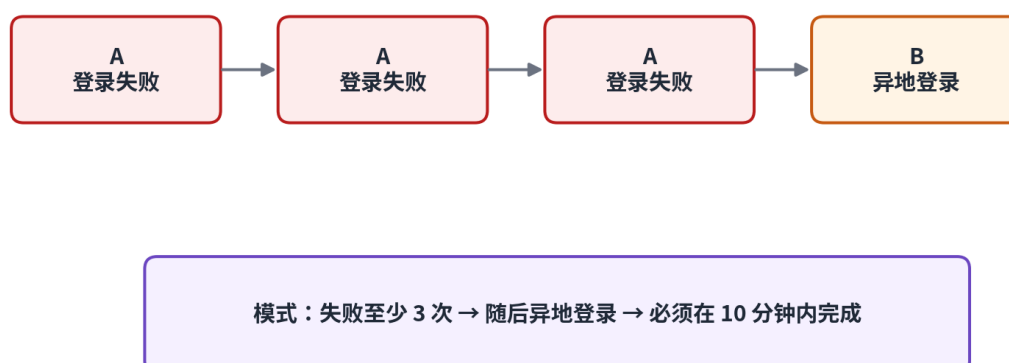
```
clicks.keyBy(Click::getUserId)
    .intervalJoin(buys.keyBy(Buy::getUserId))
    .between(Time.minutes(-5), Time.minutes(10))
    .process(new ClickBuyJoinFunction());
```

CoProcess 更适合业务规则复杂、需要自定义超时、补偿、侧输出或多种状态转换的情况。

9. 为什么还需要 CEP？

CoProcess / ProcessFunction 本质上允许你手写任意状态机。但当规则从“两类事件配对”升级成“多事件、有顺序、有次数、有否定、有时间约束的模式”，手写状态机会迅速变复杂。

CEP：从“手写状态机”升级为“描述事件模式”



CEP 关心的是“事件序列的形状”，而不是单条事件或固定窗口里的简单聚合。

图 5 CEP 的输入是事件流，输出是满足某个时序模式的事件组合

CEP（Complex Event Processing，复杂事件处理）关注的是“事件序列模式”，不是单条事件，也不是简单的窗口聚合。

10. CEP 的 Pattern API：把规则写成模式

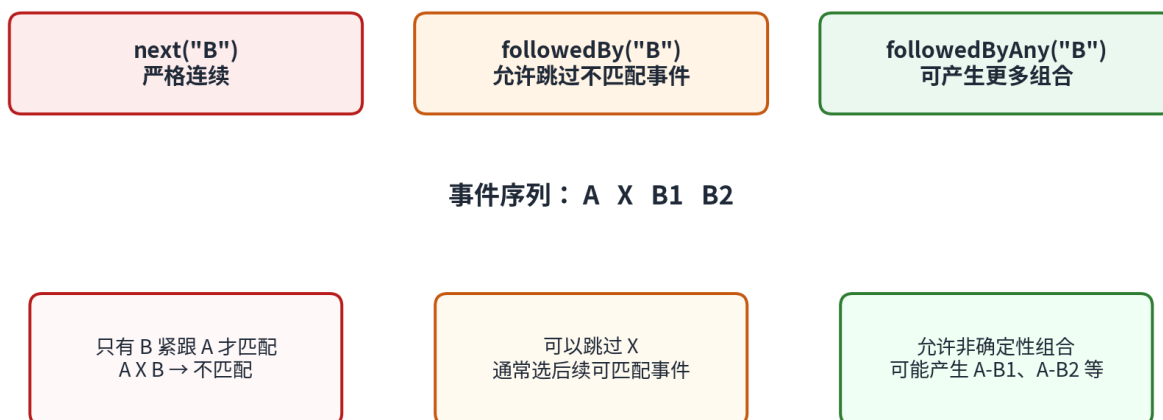
Flink CEP 的核心是 Pattern。一个 Pattern 可以描述起始事件、后续事件、条件、重复次数、时间约束，以及否定模式。

```
Pattern<LoginEvent, ?> pattern =  
    Pattern.<LoginEvent>begin("fail")  
        .where(e -> e.isFailed())  
        .timesOrMore(3)  
        .followedBy("remote")  
        .where(e -> e.isRemoteLogin())  
        .within(Duration.ofMinutes(10));
```

这段代码表达的是：先出现至少 3 次失败登录，随后出现异地登录，并且整个模式要在 10 分钟内完成。

11. next、followedBy、followedByAny：为什么不是同一个“然后”？

CEP 中“接着发生”到底有多严格？



CEP 难点之一：不是只有“条件”，还要定义事件之间的时间连续性与消费策略。

图 6 CEP 中不同连续性策略决定“中间事件能不能跳过”

在复杂模式中，“A 后面出现 B”可能有不同含义：

操作	直觉	对中间事件的态度
next("B")	B 必须紧跟 A	严格连续，不允许夹别的事件
followedBy("B")	A 之后找一个 B	允许跳过不匹配事件
followedByAny("B")	A 之后所有候选 B 都可能形成匹配	允许保留更多组合

12. 次数、可选、否定与时间约束

- oneOrMore(): 某事件至少出现一次，可重复。
- times(n): 精确出现 n 次。
- times(from, to): 出现次数落在一个范围。
- optional(): 这一段可以出现，也可以不出现。
- notNext() / notFollowedBy(): 要求某种事件不要在指定位置出现。
- within(Duration): 整个匹配必须在规定时间内完成。

CEP 的难点不只是“条件判断”，而是条件 + 顺序 + 次数 + 时间 + 事件消费策略的组合。

13. CEP 底层为什么常用 NFA 来理解？

Flink 的 Pattern 会被编译成内部的 NFA（Non-deterministic Finite Automaton，非确定有限自动机）。你不需要手写 NFA，但它能帮助理解：为什么系统可以同时保留多个“尚未完成的候选匹配”。

CEP 底层直觉：非确定有限自动机（NFA）同时保留多个候选路径

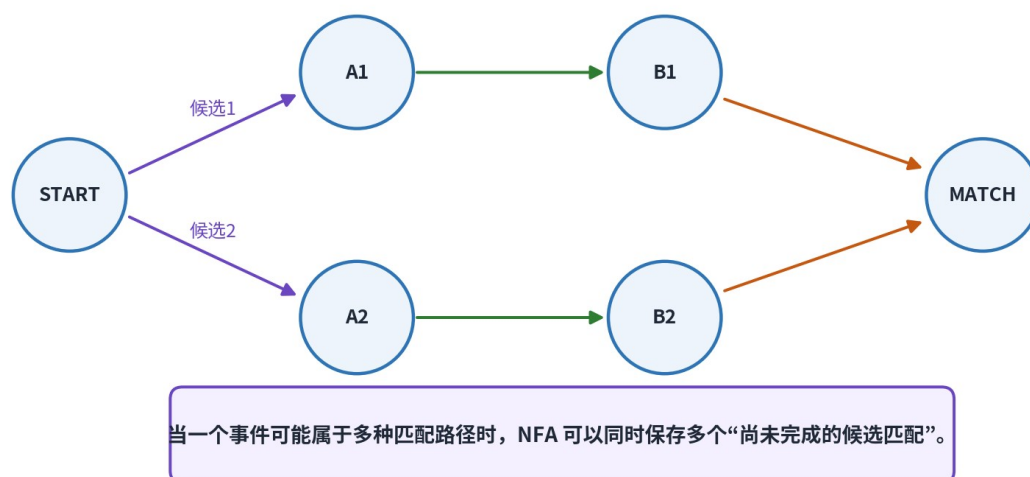


图 7 NFA 可以同时维护多条可能的匹配路径

例如 A1、A2 都可能成为同一个模式的起点，后续 B 到来时，系统可能需要同时评估多个候选分支，而不是只记住“最后一个 A”。

14. CEP 不是“无限记忆”：Pattern 必须考虑状态增长

复杂模式匹配会保留部分匹配状态。如果条件太宽松、时间范围太大、重复模式太多，候选匹配可能迅速膨胀。工程上必须考虑：

- 尽量给模式设置合理的 within 时间约束。
- 条件要足够有选择性，避免所有事件都成为候选。
- 谨慎使用 oneOrMore()、allowCombinations() 等会扩展组合数量的规则。
- 理解 AfterMatchSkipStrategy 等匹配后跳过策略，控制重复匹配。

15. 完整风控例子：失败登录 → 异地登录

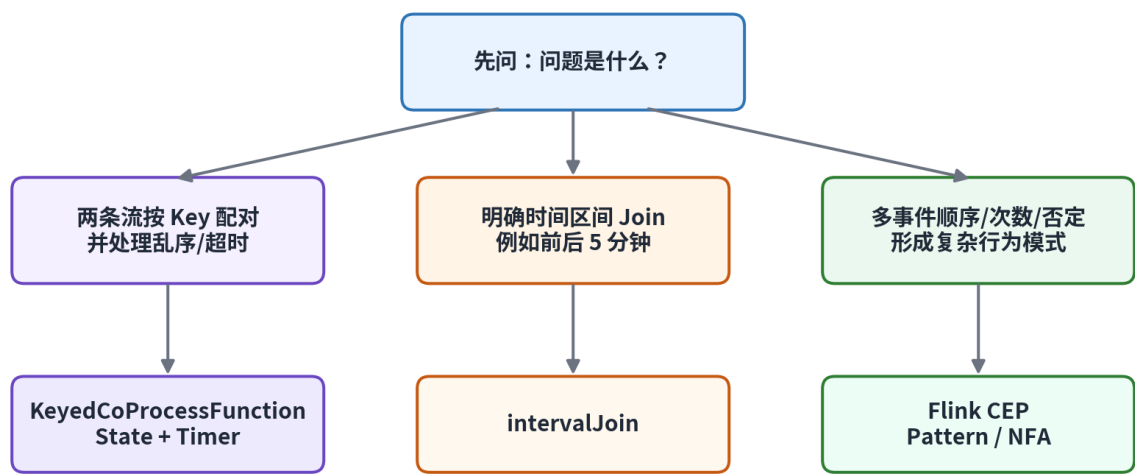
假设某用户事件序列如下：

```
10:00 FAIL Beijing
10:01 FAIL Beijing
10:02 FAIL Beijing
10:05 NORMAL_VIEW
10:07 SUCCESS NewYork
```

我们希望识别“10 分钟内至少三次登录失败，随后出现异地成功登录”。这个问题用 ProcessFunction 也能做，但需要自己维护失败次数、首个时间、地理位置、超时 Timer、状态清理。CEP 则更接近直接描述“我们要找的模式”。

16. CoProcess 还是 CEP？一张决策图

实战决策：双流匹配还是 CEP？



优先使用最简单、最能直接表达业务约束的工具。复杂度本身不是目标。

图 8 工具选择应从业务结构出发，而不是从 API 名称出发

一句话：两条流“配对并协调状态”优先想 CoProcess；多事件“顺序模式识别”优先想 CEP。

17. 常见误区

误区	为什么不对
双流一定要 Join	CoProcess 可以表达更复杂的乱序、超时和状态逻辑。
connect 等于 union	connect 保留两侧输入类型和独立处理入口；union 要求同类型并直接合并。
PAY 一定晚于 CREATE	业务发生顺序和系统到达顺序不是一回事。
CEP 就是正则表达式	相似的是“模式描述”直觉，但 CEP 还涉及事件时间、状态、匹配分支和分布式执行。
CEP 越复杂越高级	复杂规则会带来更多状态与候选路径，必须控制成本。
有 CEP 就不需要 ProcessFunction	很多简单状态机用 ProcessFunction 更透明、更容易维护。

18. 一个零依赖小实验：手工模拟双流乱序匹配

```
events = [
    ("PAY", 1002),
    ("CREATE", 1001),
    ("PAY", 1001),
    ("CREATE", 1002),
]

create_state = set()
pay_state = set()

for typ, oid in events:
    if typ == "CREATE":
        if oid in pay_state:
            print("MATCH", oid)
            pay_state.remove(oid)
        else:
            create_state.add(oid)
    else:
        if oid in create_state:
            print("MATCH", oid)
            create_state.remove(oid)
        else:
            pay_state.add(oid)

print("unmatched create:", create_state)
print("unmatched pay:", pay_state)
```

这个实验没有 Flink，但能帮助你理解 CoProcess 的核心：两侧到达顺序可以任意，状态充当“暂存区”，直到另一边到来或 Timer 宣布等待结束。

19. 练习题

5. 订单流和退款流需要按 orderId 配对，退款可能先到。你会选择 union、intervalJoin 还是 CoProcess？为什么？
6. 如果要求“用户 5 分钟内连续失败登录 3 次”，你会优先用 KeyedProcessFunction 还是 CEP？两者各有什么优缺点？
7. next("B") 和 followedBy("B") 的最大区别是什么？
8. 为什么 CEP 需要 within 这类时间约束？
9. 一个 CEP Pattern 使用 oneOrMore().allowCombinations() 后，为什么状态可能变大？

参考答案

10. 通常优先考虑 KeyedCoProcessFunction：两侧按 orderId keyBy，状态保存尚未匹配事件，Timer 清理长期未匹配记录。若时间区间非常明确，也可考虑 intervalJoin。
11. 简单“连续失败计数”用 KeyedProcessFunction 更直接；若后续加入成功/异地/设备变化/否定条件等多事件序列，CEP 更有表达力。
12. next 要求严格连续；followedBy 允许中间出现不匹配事件。
13. 限制候选匹配的生命周期，控制业务语义，也防止部分匹配状态无限保留。
14. 因为一个事件可能保留多个组合分支，候选匹配数量会增加，进而扩大状态和计算成本。

20. 本讲总结：把“事件协同”分成两层

第一层：CoProcess = 两条流 + 同一 Key + State + Timer，解决“两个来源怎样协调”。

第二层：CEP = Pattern + 条件 + 顺序 + 次数 + 时间约束 + NFA，解决“复杂事件序列怎样被识别”。

两条流“放到一起”并不只有一种方法

工具	本质	特点	典型场景
union	同类型流直接合并	无需分别识别来源	多个同构日志源
connect	不同类型流联合处理	两侧保留各自类型	订单流 + 支付流
intervalJoin	按 Key + 时间区间配对	适合明确时间范围 Join	点击 + 曝光
CoProcess	两侧事件 + State + Timer	控制力最强	乱序、超时、补偿
CEP	描述多事件时序模式	模式语言 + 状态机	风控、入侵、行为序列

选择工具的关键不是“哪个更高级”，而是：你的问题是合并、Join、状态协调，还是复杂模式识别？

图 9 回到全局：不同问题应选择不同抽象

本讲核对资料：Apache Flink 当前稳定文档中的 Process Function / Low-level Joins、
DataStream.connect / ConnectedStreams API，以及 Flink CEP Pattern / CEP Java API。